Construction Helmet Use Recognition: A Benchmarked and Explainable CNN Classification Study

Manogna Mannala
MSc Data Science, University of Europe for Applied Sciences
Berlin, Germany

***Abstract*— Hard hats are the first line of defence against head injuries on construction sites, yet checking that every worker is wearing one still depends largely on manual supervision. This study frames helmet compliance as a binary image recognition problem: given a cropped image of a worker's head, decide whether a helmet is present. The Kaggle Hard Hat Workers detection dataset was converted into 17,044 cleaned head crops (13,410 helmet, 3,634 no-helmet). A custom CNN trained from scratch (258,881 parameters) was compared with ImageNet-pretrained MobileNetV2 and ResNet50 models with new classification heads. On the held-out test set MobileNetV2 reached 97.34% accuracy, an F1-score of 0.933 and a recall of 0.926 on the safety-critical no-helmet class. The custom CNN reached 94.44% accuracy and 0.839 recall with about one-ninth of the parameters and 22% lower latency. The ResNet50 run was interrupted during training and could not be evaluated. Grad-CAM was used to inspect the custom CNN's decisions. The paper closes with the limitations of the experimental setup that should be addressed before the numbers are treated as final.**

***Index Terms*— construction safety, helmet use recognition, convolutional neural network, transfer learning, MobileNetV2, ResNet50, Grad-CAM**

I. Introduction

Construction is one of the most hazardous industries to work in. Falling objects and falls from height are common causes of serious injury, and the head is especially vulnerable. Regulations require hard hats across active work areas, but in practice compliance is checked by supervisors walking the site. This is slow, depends on who is watching, and easily misses short lapses.

Most large sites already run CCTV, so automatic visual checking is an obvious next step. Earlier studies treated the problem as object detection, locating every helmet and bare head in a full frame [1–3]. This project takes a narrower view. Assuming that heads have already been located (by an upstream detector, or from annotations as here), the remaining question is a simple one: *is this head wearing a helmet?* Treating this as image classification makes it easy to benchmark different CNN architectures fairly and to look inside their decisions.

The aims of the project were to:

- build a clean helmet/no-helmet classification dataset from a public detection dataset;
- compare a CNN trained from scratch against two transfer-learning models;
- evaluate the models with metrics that reflect the safety cost of each error type, plus inference latency;
- use Grad-CAM to check what image regions drive the predictions.

II. Related Work

Fang *et al.* [1] used Faster R-CNN on far-field surveillance video and showed that non-hardhat-use can be detected even when workers are small in the frame. Wu *et al.* [2] published a hardhat benchmark that also labels helmet colour, and improved an SSD detector for the task. Nath *et al.* [3] compared three YOLO-based approaches to PPE detection, one of which detects workers first and then classifies their PPE with a separate CNN. That two-stage design is the closest to the setting studied here.

Transfer learning [4] re-uses features learned on a large source dataset such as ImageNet [5]. ResNet50 [6] and MobileNetV2 [7] represent the accurate-but-heavy and the efficient ends of the range of common backbones. Batch normalisation [8] and dropout [9] are standard in small CNNs trained from scratch. Grad-CAM [10] produces class-specific heat maps from the gradients of the last convolutional layer and is widely used to sanity-check image classifiers.

III. Dataset and Preparation

A. Source dataset

The Hard Hat Workers dataset [11] has 5,000 site images, each with a Pascal-VOC XML file. Scanning all annotations gave 18,966 `helmet` boxes, 5,785 `head` boxes (head without a helmet) and 751 `person` boxes.

B. From detection boxes to classification crops

Each `helmet` box was mapped to the class *helmet* and each `head` box to *no_helmet*. `person` boxes were dropped because a whole-body box says nothing direct about the head. Every remaining box was cropped with 10% padding on each side, and boxes smaller than 20 px in either dimension were skipped.

```
CLASS_MAP = {"helmet": "helmet", "head": "no_helmet", "
    person": None}
crop_summary = voc_crop.crop_dataset(
   ANNOTATIONS_DIR, IMAGES_DIR, CROPPED_DIR, CLASS_MAP,
   min_box_size=20, padding_frac=0.10)
```

The mapped boxes total $18{,}966 + 5{,}785 = 24{,}751$. Of these, 7,673 were too small, leaving

$$24{,}751 - 7{,}673 = 17{,}078 \text{ crops},$$

made up of 13,443 helmet and 3,635 no_helmet crops. So about $7{,}673/24{,}751 = 31.0\%$ of usable boxes were lost to the size fil-

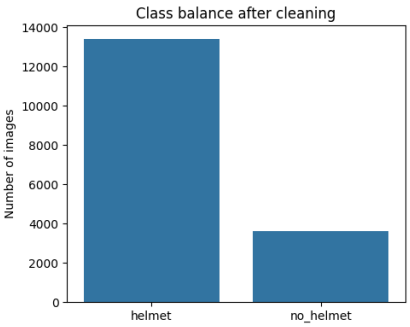


Fig. 1. Class balance of the cleaned crop dataset.

ter. This is a large share and reflects how many workers appear far from the camera.

C. Cleaning

A conservative cleaning pass flagged near-duplicates with perceptual hashing [12] and blurry crops with the variance of the Laplacian [13], using a blur threshold of 15. A dry run was checked first. Flagged files were moved to a review folder rather than deleted. The pass removed 34 crops (27 duplicates, 7 blurry), giving the final dataset:

$$13{,}410\ \text{helmet} + 3{,}634\ \text{no_helmet} = 17{,}044\ \text{crops}.$$

D. Class imbalance

The imbalance ratio is

$$\frac{13{,}410}{3{,}634} = 3.69,$$

and the no-helmet share is $3{,}634/17{,}044 = 21.32\%$. A trivial model that always answers *helmet* would therefore score $1 - 0.2132 = 78.68\%$ accuracy while catching no violations at all. This is the reason accuracy alone is not a useful headline metric here (Fig. 1).

E. Splitting

Crops were resized to 160×160 and loaded with Keras `image_dataset_from_directory` using seed 42. Labels follow alphabetical folder order, so *helmet* $= 0$ and *no_helmet* $= 1$; the no-helmet class is therefore the positive class in all metrics below. A 30% holdout was taken first:

$$\lfloor 0.30 \times 17{,}044 \rfloor = 5{,}113\ \text{holdout},$$
$$17{,}044 - 5{,}113 = 11{,}931\ \text{train}.$$

With batch size 32 the holdout has $\lceil 5{,}113/32 \rceil = 160$ batches. The first $160/2 = 80$ batches ($80 \times 32 = 2{,}560$ crops) form the validation set and the rest form the test set:

$$5{,}113 - 2{,}560 = 2{,}553\ \text{test crops}.$$

F. Augmentation

Training batches were augmented with horizontal flips, small rotations (factor 0.05, about $\pm 18°$), and brightness and contrast jitter of $\pm 15\%$. Vertical flips were deliberately left out because an upside-down worker is not a realistic view. Fig. 2 shows a sample.



Fig. 2. Augmented training crops.

IV. Methodology

A. Custom CNN

The baseline network is a four-stage CNN trained from scratch on inputs scaled to $[0, 1]$. Each stage is a 3×3 convolution with ReLU, batch normalisation and 2×2 max-pooling. Global average pooling feeds a 128-unit dense layer, with dropout of 0.4 before and after it, and a single sigmoid output.

```
x = layers.Conv2D(32, 3, padding="same", activation="relu")(
    inputs)
x = layers.BatchNormalization()(x)
x = layers.MaxPooling2D()(x)
# ... repeated with 64, 128, 128 filters ...
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(dropout)(x)
x = layers.Dense(128, activation="relu")(x)
x = layers.Dropout(dropout)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
```

Feature map sizes. With `padding="same"` and stride 1, each convolution keeps the spatial size. Each 2×2 pool with stride 2 halves it:

$$160 \to 80 \to 40 \to 20 \to 10.$$

The final map is $10 \times 10 \times 128$, and global average pooling reduces it to a 128-dimensional vector.

Parameter count. A convolution with bias has $k^2 C_{\text{in}} C_{\text{out}} + C_{\text{out}}$ parameters. A batch-norm layer has $4C$: γ and β are trained, while the moving mean and variance are not. Working through each layer:

$$\begin{aligned}
\text{Conv}_1 &= 9 \cdot 3 \cdot 32 + 32 = 864 + 32 = 896 \\
\text{Conv}_2 &= 9 \cdot 32 \cdot 64 + 64 = 18{,}432 + 64 = 18{,}496 \\
\text{Conv}_3 &= 9 \cdot 64 \cdot 128 + 128 = 73{,}728 + 128 = 73{,}856 \\
\text{Conv}_4 &= 9 \cdot 128 \cdot 128 + 128 = 147{,}456 + 128 = 147{,}584 \\
\text{BN (all)} &= 4\,(32 + 64 + 128 + 128) = 4 \cdot 352 = 1{,}408 \\
\text{Dense}_{128} &= 128 \cdot 128 + 128 = 16{,}512 \\
\text{Dense}_1 &= 128 \cdot 1 + 1 = 129
\end{aligned}$$

Summing:

$$896 + 18{,}496 + 73{,}856 + 147{,}584 + 1{,}408 + 16{,}512 + 129 = 258{,}881,$$

which matches the Keras summary. The non-trainable part is the batch-norm statistics, $2 \cdot 352 = 704$, leaving 258,177 trainable parameters.

B. Transfer-learning models

MobileNetV2 [7] and ResNet50 [6] were loaded with ImageNet weights and without their top layers. Each backbone's own `preprocess_input` function sits inside the model as a `Lambda` layer, so the models take raw $[0, 255]$ pixels. The head is global average pooling, dropout of 0.3 and a sigmoid unit.

```
base_model = backbone_cls(input_shape=input_shape,
                  include_top=False, weights="imagenet")
base_model.trainable = False
x = layers.Lambda(preprocess_fn, name="preprocess")(inputs)
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
```

The new head adds $1{,}280 + 1 = 1{,}281$ parameters to MobileNetV2, giving 2,259,265 in total. It adds $2{,}048 + 1 = 2{,}049$ to ResNet50, giving 23,589,761 in total.

The backbones stayed frozen throughout training, so only the new head was learned. Fine-tuning the upper backbone layers is left for future work.

C. Loss and optimisation

All models minimise binary cross-entropy. For one sample with label $y \in \{0, 1\}$ and predicted probability $p = \sigma(z)$ of *no_helmet*,

$$\mathcal{L}(y, p) = -\left[\, y \log p + (1 - y) \log(1 - p) \,\right]. \tag{1}$$

For example, a bare head ($y = 1$) predicted at $p = 0.9$ costs $-\log 0.9 = 0.105$. Predicted at $p = 0.1$, it costs $-\log 0.1 = 2.303$, about 22 times more.

Optimisation used Adam [14] at learning rate 10^{-3} for all three models, with a batch size of 32 and a two-epoch budget. Early stopping on validation loss (patience 5, best weights restored) and checkpointing were enabled. The model is implemented in TensorFlow [15].

D. Evaluation metrics

Treating *no_helmet* as positive, with the counts TP, FP, FN and TN taken from the confusion matrix:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \tag{2}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \tag{3}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{4}$$

On a site, a false negative (a bare head marked as safe) is the expensive error, so recall on no-helmet carries the most weight. ROC-AUC summarises how well the model ranks bare heads above helmets across all thresholds. Mean single-image inference time was also measured over 50 test crops after 5 warm-up calls.

TABLE I
TRAINING AND VALIDATION FIGURES PER EPOCH

Model	Ep.	Train loss	Train acc.	Val loss	Val acc.
Custom CNN	1	0.3120	0.8655	0.3884	0.7988
	2	0.2106	0.9195	0.1526	0.9453
MobileNetV2	1	0.2170	0.9129	0.1044	0.9668
	2	0.1323	0.9492	0.0910	0.9684
ResNet50	1	0.1515	0.9411	0.0771	0.9750
	2	0.0894	0.9653	–	–

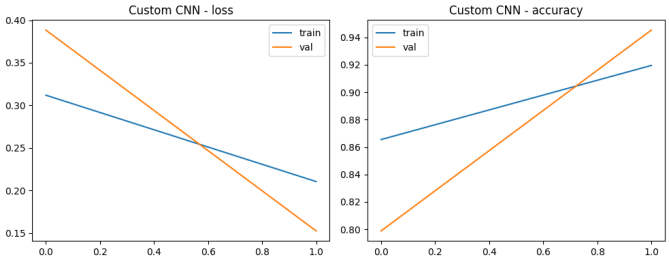


Fig. 3. Custom CNN loss and accuracy over the two training epochs.

V. RESULTS

A. Training behaviour

Each model was trained for two epochs on the Colab CPU runtime (no GPU was attached). Table I lists the epoch-level figures.

The custom CNN improved sharply between the two epochs. Validation accuracy rose from 79.88% to 94.53%, and validation loss fell from 0.3884 to 0.1526, a drop of

$$\frac{0.3884 - 0.1526}{0.3884} = \frac{0.2358}{0.3884} = 60.7\%.$$

Fig. 3 shows both curves still moving steeply at the end, so the model had not converged.

MobileNetV2 started far higher: 96.68% validation accuracy after one epoch with only its head trained. The second epoch added only 0.16 points (96.84%), suggesting the frozen features had already done most of the work. For MobileNetV2 the validation loss stays below the training loss, which is expected, since dropout and augmentation are active only during training.

ResNet50 reached the best validation accuracy of all after one epoch (97.50%). Its second epoch finished the training pass (train accuracy 0.9653) but the run was interrupted by hand before validation completed, because of session time. As a result, ResNet50 never entered the evaluation loop and has no test results.

B. Test-set performance

Table II summarises test performance, and Fig. 4 shows the confusion matrices.

TABLE II
TEST-SET RESULTS (POSITIVE CLASS = NO_HELMET, THRESHOLD 0.5)

Metric	Custom CNN	MobileNetV2
Accuracy	0.9444	**0.9734**
Precision	0.8833	**0.9407**
Recall	0.8394	**0.9261**
F1-score	0.8608	**0.9333**
ROC-AUC	0.9773	**0.9925**
Parameters	**258,881**	2,259,265
Latency (ms)	**116.5**	149.6

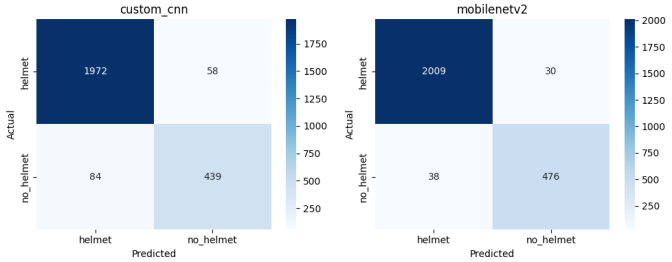


Fig. 4. Test-set confusion matrices (rows: actual, columns: predicted).

Working from the confusion matrices. For the custom CNN, $TP = 439$, $FN = 84$, $FP = 58$ and $TN = 1{,}972$:

$$\text{Precision} = \frac{439}{439+58} = \frac{439}{497} = 0.8833$$

$$\text{Recall} = \frac{439}{439+84} = \frac{439}{523} = 0.8394$$

$$F_1 = \frac{2 \times 0.8833 \times 0.8394}{0.8833+0.8394} = \frac{1.4829}{1.7227} = 0.8608$$

$$\text{Accuracy} = \frac{1{,}972+439}{2{,}553} = \frac{2{,}411}{2{,}553} = 0.9444$$

For MobileNetV2, $TP = 476$, $FN = 38$, $FP = 30$ and $TN = 2{,}009$:

$$\text{Precision} = \frac{476}{476+30} = \frac{476}{506} = 0.9407$$

$$\text{Recall} = \frac{476}{476+38} = \frac{476}{514} = 0.9261$$

$$F_1 = \frac{2 \times 0.9407 \times 0.9261}{0.9407+0.9261} = \frac{1.7424}{1.8668} = 0.9333$$

$$\text{Accuracy} = \frac{2{,}009+476}{2{,}553} = \frac{2{,}485}{2{,}553} = 0.9734$$

To put the custom CNN's accuracy in context, an always-helmet model scores $2{,}030/2{,}553 = 79.51\%$ on its test sample. The custom CNN improves on that by

$$94.44 - 79.51 = 14.93 \text{ percentage points},$$

and recovers 84% of bare heads, so it has learned a genuinely useful decision rule even after two epochs.

MobileNetV2 is better on every quality metric. It misses 38 bare heads against the custom CNN's 84, a reduction of $(84-38)/84 = 54.8\%$. It also raises 30 false alarms against 58, a reduction of $(58-30)/58 = 48.3\%$. The two models' ROC-AUC values (0.977 and 0.993) are both high, and their ROC curves (Fig. 5) both bend sharply towards the top-left corner.

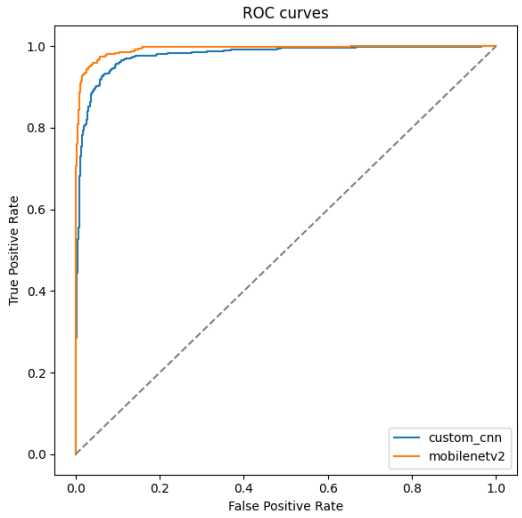


Fig. 5. ROC curves on the test set.

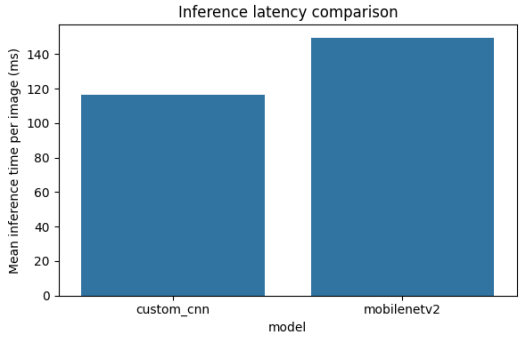


Fig. 6. Mean single-image inference time (CPU).

C. Model size and latency

The custom CNN has

$$\frac{258{,}881}{2{,}259{,}265} = 0.115$$

of MobileNetV2's parameters, roughly one-ninth. Its mean single-image latency is 116.5 ms against 149.6 ms (Fig. 6), which is

$$\frac{149.6 - 116.5}{149.6} = \frac{33.1}{149.6} = 22.1\%$$

faster. These times were measured on CPU with one Keras `predict()` call per image. Each call carries a large fixed overhead, so the absolute numbers overstate true model cost, and the gap between the models would be expected to change under batched GPU inference.

D. Explainability with Grad-CAM

Grad-CAM [10] weights each feature map A^k of the last convolutional layer by the average gradient of the output y with



Fig. 7. Grad-CAM for the custom CNN. Top: correct predictions. Bottom: bare heads predicted as helmet (false negatives).

respect to it:

$$\alpha_k = \frac{1}{Z}\sum_i\sum_j \frac{\partial y}{\partial A^k_{ij}}, \qquad L = \mathrm{ReLU}\Big(\sum_k \alpha_k A^k\Big), \tag{5}$$

where Z is the number of spatial positions. The map L is then normalised to $[0,1]$ and resized to the input.

Fig. 7 shows maps for the custom CNN. In correct predictions, much of the activation sits along the borders of the crop rather than on the helmet shell. In the missed bare heads, the activation falls on parts of the face and on background. This suggests the custom CNN partly relies on cues from the padding region and image edges, not only on the head itself.

Grad-CAM for MobileNetV2 failed with a `KeyError`, which the notebook caught and reported. The code builds the Grad-CAM sub-model from a layer inside the nested backbone, and Keras 3 does not connect that layer to the outer model's graph, so no heat maps were produced for the transfer model.

E. Error analysis

Looking at the misclassified test crops in the Grad-CAM rows, the main failure patterns are:

- dark or low-contrast crops where the outline of the head is hard to see;
- bare heads with dark hair against a dark background, which can resemble a dark helmet;
- heads seen from behind or at a steep angle, where neither a face nor a helmet brim is clearly visible;
- low-resolution crops upsampled to 160×160, which blurs the helmet edge.

VI. Discussion and Limitations

Even with a two-epoch budget, both models learned the task well. Pretrained ImageNet features give MobileNetV2 a clear lead on every quality metric, while the custom CNN is smaller and faster and is still improving. For a deployment that must catch as many violations as possible, MobileNetV2 is the better choice. Where hardware is very constrained, the custom CNN is a reasonable option, provided it is trained for longer.

Several aspects of the experiment limit how far the exact numbers can be trusted, and they should be fixed in a follow-up run:

- **Very short training on CPU.** No GPU was attached, so epochs took 6–26 minutes and only two were possible. The custom CNN's curves had not flattened.
- **ResNet50 not evaluated.** The run was interrupted during its second epoch, so the comparison covers only two of the three planned models.
- **Validation/test split.** The validation and test sets come from the same holdout dataset through `take()` and `skip()`. Keras reshuffles that dataset each time it is iterated, so the two subsets are not fixed and can overlap. Evidence for this is that the test class counts differ between models (523 against 514 no-helmet crops), even though the test set should be identical. Setting `shuffle=False` on the holdout, or splitting the files beforehand, would fix this.
- **Crop-level split.** Crops are split at random rather than by source image, so crops of the same scene, and sometimes the same worker, can appear in both training and test data.
- **No class weighting or threshold tuning.** All results use a fixed 0.5 threshold and an unweighted loss on 3.69 : 1 data. Either change could raise no-helmet recall further.
- **Grad-CAM for transfer models.** The nested-backbone issue needs fixing before the transfer models can be inspected.

VII. Conclusion and Future Work

This project built an end-to-end pipeline for construction helmet use recognition, covering conversion of a detection dataset into cleaned head crops, a custom CNN and two transfer-learning models, evaluation with safety-relevant metrics and latency, and Grad-CAM inspection. MobileNetV2 gave the best results, with 97.34% accuracy and 0.926 no-helmet recall. The custom CNN reached 94.44% accuracy and 0.839 recall with about one-ninth of the parameters and 22% lower latency.

Next steps are to train on a GPU for more epochs, complete the ResNet50 evaluation, fix the validation/test split and split by source image, add class weights and tune the decision threshold on validation data, repair Grad-CAM for the transfer models, and pair the classifier with a head detector for end-to-end monitoring of site video.

References

[1] Q. Fang, H. Li, X. Luo, L. Ding, H. Luo, T. M. Rose, and W. An, "Detecting non-hardhat-use by a deep learning method from far-field surveillance videos," *Automation in Construction*, vol. 85, pp. 1–9, 2018.

[2] J. Wu, N. Cai, W. Chen, H. Wang, and G. Wang, "Automatic detection of hardhats worn by construction personnel: A deep learning approach and benchmark dataset," *Automation in Construction*, vol. 106, p. 102894, 2019.

[3] N. D. Nath, A. H. Behzadan, and S. G. Paal, "Deep learning for site safety: Real-time detection of personal protective equipment," *Automation in Construction*, vol. 112, p. 103085, 2020.

[4] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010.

[5] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 248–255, 2009.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

[7] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520, 2018.

[8] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Machine Learning (ICML)*, pp. 448–456, 2015.

[9] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.

[10] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, pp. 618–626, 2017.

[11] Make ML, "Hard hat workers dataset." Kaggle, uploaded by andrewmvd. `https://www.kaggle.com/datasets/andrewmvd/hard-hat-detection`. Accessed: Oct. 2026.

[12] C. Zauner, "Implementation and benchmarking of perceptual image hash functions," Master's thesis, Upper Austria University of Applied Sciences, Hagenberg Campus, 2010.

[13] J. L. Pech-Pacheco, G. Cristóbal, J. Chamorro-Martínez, and J. Fernández-Valdivia, "Diatom autofocusing in brightfield microscopy: a comparative study," in *Proc. 15th Int. Conf. Pattern Recognition (ICPR)*, vol. 3, pp. 314–317, 2000.

[14] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[15] M. Abadi, P. Barham, J. Chen, *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, pp. 265–283, 2016.